Lab 04: Controlled Empirical User Study Plan

Project: Weber State University "Register for Classes" (eWeber student portal)
Researcher: Kaden Donaldson
Study type: Controlled, within-subjects usability experiment (2 conditions, counterbalanced)

Supporting file (GitHub repo, `experiment_design/`)	Purpose
`README.md`	This consolidated report (Lab04_Experiment_Plan)
`research_protocol.md`	Step by step session procedure, moderator script, checklists
`variable_mapping.md`	IV / DV / control tables and hypothesis to metric to test mapping
`counterbalancing_matrix.csv`	Participant to sequence assignment (Latin square)
`data_dictionary.csv`	Definition, type, unit, and valid range of every recorded variable
`data_collection_sheet.csv`	Blank trial-level sheet, pre-filled with the session schedule

---

Step 1: Interface Task Selection and Empirical Problem Framing

Target System Flow

Keyboard-only course registration with full-section recovery in the Register for Classes app. The participant must (1) search for a specified course, (2) add a specified section to their cart, (3) submit registration, and (4) recover when a chosen section turns out to be full by finding and adding an open section of the same course.

This is a high-stakes flow. Registration happens in short, competitive windows, and a failed or delayed add can cost a student a required seat, push back graduation, or affect financial aid eligibility tied to credit load.

Empirical Rationale: the Bottleneck

My Lab 03 Cognitive Audit and Phase 1 Usability Audit found two concrete problems in the current interface:

1. **Keyboard focus skips the results.** When tabbing through search results, focus jumps past the listed sections and their **Add** buttons entirely. A keyboard-only user (motor impairment, screen reader user, power user, broken trackpad) cannot add a class without a mouse. This fails WCAG 2.1 SC 2.1.1 (Keyboard) and 2.4.3 (Focus Order).
2. **The full-class error is vague.** Trying to add a full section produces a generic error that does not say the section is full or what to do next. This fails WCAG 2.1 SC 3.3.1 (Error Identification) and 3.3.3 (Error Suggestion), and it pushes the user into trial and error.

How the Redesign Addresses It

The refactored interface (built from my Case Study 2 HTML/CSS/JS prototype and the Phase 2 tab order annotations) changes only these behaviors:

Problem	Baseline (Interface A)	Refactor (Interface B)	Targeted outcome
Add buttons unreachable by Tab	Click handlers on non-focusable elements, focus skips results	Native `<button>` elements in logical DOM order, 3px visible focus ring (SC 2.4.7)	Shorter time on task, fewer keyboard dead ends
Vague full-class error	Generic error, no cause, no next step	Specific message ("CS 2420 section 001 is full, 0 of 40 seats open") plus focusable links to open sections, announced through an `aria-live` region (SC 4.1.3)	Fewer errors, higher recovery success, lower cognitive load

Everything else (colors, fonts, layout, course data, page count) is identical between A and B so that any measured difference can be attributed to the manipulated behaviors.

Step 2: Hypothesis Formulation

All tests are two-condition comparisons within the same participants, alpha = .05, with Holm-Bonferroni correction across the four primary hypotheses.

#	Directional hypothesis (H1)	Null hypothesis (H0)
1	Participants using **Interface B** will complete the keyboard-only add-and-submit task (Task 1) in significantly **less time** than when using Interface A.	There is no statistically significant difference in Task 1 completion time between Interface A and Interface B.
2	Participants using **Interface B** will commit significantly **fewer errors** across both tasks than when using Interface A.	There is no statistically significant difference in total error count between Interface A and Interface B.
3	A significantly **higher proportion** of participants will successfully recover from the full-section error (Task 2) using **Interface B** than using Interface A.	There is no statistically significant difference in Task 2 success rate between Interface A and Interface B.

#	Directional hypothesis (H1)	Null hypothesis (H0)
4	Participants will rate **Interface B** significantly **higher** in perceived usability (SUS) than Interface A.	There is no statistically significant difference in SUS scores between Interface A and Interface B.

Secondary (exploratory) hypothesis: SEQ ratings will be higher for B on each task, and keystroke counts will be lower for B on Task 1.

Step 3: Variable Formulation and Operationalization

Independent Variables

IV	Type	Levels
Interface_Condition (primary)	Within-subjects, nominal	**A** = Baseline Control (replica of current Register for Classes behavior); **B** = Refactored Accessible Treatment
Task_Type	Within-subjects, nominal (blocking factor, not compared across)	**T1** = keyboard-only search, add, submit; **T2** = full-section error recovery
Task_Set (counterbalanced nuisance factor)	Within-subjects, nominal	**X** and **Y**: two isomorphic scenarios with different course codes and CRNs of equal difficulty (same number of results, same position of the target row, same number of open alternatives)

Dependent Variables

Construct	Variable	Operational definition	Unit / scale
Efficiency	`Task_Completion_Time_sec`	Time from the participant pressing Enter on the "Start Task" button to the confirmation screen appearing (T1) or the open alternative section appearing in the cart (T2). Logged automatically by the prototype with `performance.now()`; screen recording is the backup. Capped at 300 s; a timeout is scored 300 s and Task_Success_Binary = 0.	Seconds, ratio, 0.1 s precision
Effectiveness	`Task_Success_Binary`	1 = task end state reached within 300 s without moderator help and with the correct section in the cart. 0 = timeout, give up, moderator assist, or wrong section submitted.	0 / 1

Construct	Variable	Operational definition	Unit / scale
Effectiveness	`Error_Count_Total`	Count of discrete error events per task, coded from the recording using the codebook below.	Integer, 0 or more
Satisfaction	`SEQ_Rating`	Single Ease Question after each task: "Overall, how difficult or easy was this task to complete?"	1 (Very Difficult) to 7 (Very Easy)
Satisfaction	`SUS_Score`	10-item System Usability Scale after each condition. Score = ((sum of (odd item − 1)) + (sum of (5 − even item))) × 2.5	0 to 100
Efficiency (secondary)	`Keystroke_Count`	Total key presses logged during the task. Measures how much navigation effort the focus order costs.	Integer

Error codebook (`Error_Count_Total`):

Code	Error event	Example
E1	Wrong item activated	Adds the wrong section or wrong course
E2	Focus dead end	Participant tabs past the target and must reverse or restart (counted once per occurrence, not per key press)
E3	Wrong page / off-path navigation	Leaves the registration flow to an unrelated page
E4	Failed submission attempt	Submits and receives an error they then have to fix
E5	Misinterpreted feedback	Repeats the same failing action after an error message (for example, re-adding the full section)
E6	Mouse use attempt	Reaches for or asks for the mouse during a keyboard-only task

Two coders (me plus a classmate) code 25% of recordings independently; Cohen's kappa of .80 or higher is required before I code the rest alone.

Confounding Variables and Controls

Confound	Risk	Control strategy
Prior familiarity with eWeber	Experienced registrants are faster on A regardless of design	Screening survey records registration terms completed; participants are stratified (novice: 0 to 2 terms, experienced: 3 or more) and balanced across the four sequences. Familiarity is recorded as a covariate.
Prior exposure to the redesign	Anyone who saw my prototype already knows Interface B	Exclude everyone who took part in my Phase 1 interviews, Phase 2 card sort, or Case Study 2 review.

Confound	Risk	Control strategy
Keyboard proficiency	Fast typists or screen reader users navigate faster overall	Screening question on keyboard-only habits; within-subjects design means each person is their own control.
Input hardware	Different keyboards, trackpads, or laptops change speed	One lab laptop, one external full-size USB keyboard, mouse physically removed during tasks.
Display and browser	Zoom, resolution, or extensions change layout and tab order	1920×1080 monitor at 100% zoom, same Chrome version, fresh guest profile, no extensions, prototype run locally (no network latency).
Ambient glare and noise	Glare slows reading, noise breaks attention	Same windowless or blinds-closed study room, fixed overhead lighting, door closed, phone silenced.
Fatigue	Second condition performed worse due to tiredness	Session capped at 45 minutes, 3 minute break between conditions, counterbalanced order.
Learning / practice effects	Second condition performed better due to practice	Latin square counterbalancing of interface order and task set (Step 4), plus a neutral keyboard warm-up before the first condition.
Task content	One course scenario easier than the other	Isomorphic task sets X and Y, pilot tested with 2 people to confirm similar times; task set crossed with interface in the Latin square.
Experimenter bias	Moderator tone or hints favor B	Word for word script, moderator does not know which condition is "new" from the participant's view (both labeled "Version 1 / Version 2"), no hints except a scripted assist which counts as failure.
Real registration risk	Using the live portal could change real enrollments	Both interfaces are offline, high-fidelity prototypes with fake course data.

Step 4: Experimental Design Architecture and Counterbalancing

Subject Design: Within-Subjects

Justification:

- Individual differences in keyboard skill and registration experience are large. A within-subjects design removes that variance from the comparison because every participant uses both A and B.
- It needs fewer participants for the same statistical power, which is realistic for a class project.

- Both tasks are short (under 5 minutes each), so doing both conditions in one 45 minute session is feasible without heavy fatigue.

The main risk of within-subjects is learning and carryover, which is handled below.

Counterbalancing Scheme: 2×2 Latin Square

Interface order **and** task set order are both balanced. Each participant gets one of four sequences:

Sequence	Period 1	Period 2	Participants
S1	A with Task Set X	B with Task Set Y	P01, P05, P09, P13
S2	A with Task Set Y	B with Task Set X	P02, P06, P10, P14
S3	B with Task Set X	A with Task Set Y	P03, P07, P11, P15
S4	B with Task Set Y	A with Task Set X	P04, P08, P12, P16

Properties:

- A comes first for exactly half of participants (8) and B first for the other half (8).
- Each task set appears first for half of participants.
- Each interface is paired with each task set equally often (8 times each).

Within each period, T1 is always done before T2 because T2 starts from the cart page reached in T1.

Assignment (stratified randomization): Slots P01 to P08 are reserved for novice registrants (0 to 2 terms) and P09 to P16 for experienced registrants (3 or more terms), so every sequence gets exactly 2 novices and 2 experienced participants. Within each stratum, the order in which slots are filled is set by a pre-generated random permutation (Python `random.shuffle`, seed 4110), so arrival order cannot bias which sequence a person gets. The full matrix, including the fill order, is in `counterbalancing_matrix.csv`.

Participant Sampling Plan

Target personas:

1. **"Deadline Dana"**: undergraduate who registers the moment their window opens and is under time pressure.
2. **"Keyboard Kai"**: student who relies on the keyboard, either by preference or because of a motor or vision limitation.
3. **"First Term Felix"**: new or transfer student who has registered 0 to 2 times.

Inclusion criteria:

- Currently enrolled Weber State student, age 18 or older
- Has used, or will need to use, Register for Classes

- Can use a standard keyboard and read English text on screen at 100% zoom
- Gives informed consent

Exclusion criteria:

- Took part in my Phase 1 interviews, Phase 2 card sort, or saw my Case Study 2 prototype
- Is a member of the HCI course team grading this work, or someone with a close personal relationship to me (avoids social desirability bias)
- Works on the eWeber/Banner development team

Sample size rationale:

- **Planned N = 16** (4 per sequence). Because the design is within-subjects, every participant contributes to both conditions, giving **16 observations per condition**, well above the N ≥ 5 per condition minimum.
- N must be a multiple of 4 to keep the Latin square perfectly balanced.
- A power check for a two-tailed paired t-test at alpha = .05 shows N = 16 gives about 85% power to detect a large within-subject effect ($d_z = 0.8$), compared with about 71% at N = 12 and 50% at N = 8. Large effects are realistic here because Interface A blocks keyboard access almost completely.
- Five participants is enough for a formative "find the problems" test, but this is a summative comparison, so a larger sample is needed for hypothesis testing.
- Two extra people are recruited as backups in case of no-shows or a data failure (for example, a recording crash).

Planned Analysis

Hypothesis	DV	Test	Fallback
H1	Task_Completion_Time_sec (T1)	Paired t-test on log-transformed time	Wilcoxon signed-rank if residuals are not normal (Shapiro-Wilk $p < .05$)
H2	Error_Count_Total (T1 + T2)	Wilcoxon signed-rank	Poisson GLMM if overdispersion is low
H3	Task_Success_Binary (T2)	Exact McNemar test	None needed
H4	SUS_Score	Paired t-test	Wilcoxon signed-rank
Exploratory	SEQ_Rating, Keystroke_Count	Wilcoxon signed-rank	

Order and carryover checks: a mixed model with Interface, Period, and Sequence as fixed effects and Participant as a random effect. If the Sequence (carryover) term is significant, I report Period 1 data alone as a between-subjects comparison (see Reflection 2).

Each result is reported with an effect size and 95% confidence interval, and judged against the practical significance thresholds in Reflection 1.

Step 5: Ethical Protocol Safeguards

Ethics Safeguards

- **Voluntary:** participants may skip any task or stop at any time with no penalty and no need to explain.
- **No real records touched:** prototypes use fake student data; no real eWeber login, W number, or schedule is used.
- **Privacy:** data is stored under participant IDs (P01 to P16). The ID to name key is kept in a separate encrypted file and deleted at the end of the semester. Screen recordings capture only the screen, never faces or voices unless the participant opts in to audio.
- **Minimal risk:** tasks are routine computer use. Breaks are offered, and frustration is watched for.
- **Accessible materials:** consent form available in large print and as an accessible digital document; the moderator reads it aloud on request.
- **Institutional review:** this is a class exercise. If any data were used beyond the course, the protocol would be submitted to the Weber State IRB first.

Participant Briefing Script

> Hi, thanks for coming in. My name is Kaden, and I'm a computer science student at Weber State. Today I'm testing two versions of a class registration screen to see which one works better.
>
> **I want to be really clear about one thing: we are testing the software, not you.** If something is confusing or you get stuck, that tells me the design has a problem. That is exactly what I need to find out. There are no wrong answers, and nothing you do today reflects on your intelligence or your ability.
>
> Here's what will happen. You'll use two versions of the registration screen, called Version 1 and Version 2. With each one, you'll do two short tasks, like adding a class. For these tasks, please use **only the keyboard**. I've taken the mouse away so we can study keyboard navigation. Each task has a 5 minute limit, and if you hit it, that's completely fine. It just means the design needs work.
>
> After each task, I'll ask you one quick question about how easy it was, and after each version, a short 10 question survey. The whole session takes about 40 to 45 minutes, with a short break in the middle.
>
> The computer will record the screen and your key presses, but **not your face**, and your name will never be attached to your results. Everything is stored under an ID number. None of this uses your real student account or your real schedule; all the classes and data are fake.

> Taking part is completely voluntary. You can skip anything, take a break, or stop at any time, and you don't have to give a reason. Stopping won't affect your grades or your relationship with Weber State in any way.
>
> I won't be able to help much during the tasks because I want to see how the design works on its own, but you're welcome to think out loud. Do you have any questions before we start? If you're comfortable continuing, please read and sign the consent form, or tell me you agree and I'll note it.

Debriefing (end of session)

> Thank you. To explain a bit more now that you're done: one of the versions copied how the current registration system behaves, and the other was my redesign. I didn't say which was which beforehand so it wouldn't affect how you used them. Your results will help show whether the changes actually help students. Do you have any questions, or anything you'd like to tell me about either version?

The full session procedure is in `research_protocol.md`.

Post-Mortem Reflection

1. Why distinguish statistical significance ($p < .05$) from practical significance?

A p-value only tells me how unlikely my result would be if there were truly no difference between the interfaces. It says nothing about whether the difference is big enough to matter. With a large enough sample, a 1.5 second improvement in registration time could come out "significant," but no student would ever notice it, and it would not be worth the cost of rebuilding the interface. The reverse also happens: with only 16 people, a real and important improvement can miss $p < .05$ because the sample is small, and if I only looked at the p-value, I might throw out a good redesign.

That is why I set practical thresholds before collecting data, based on what matters during a real registration window:

Metric	Practically meaningful if
Task 1 time	At least 20% faster, or 30 seconds saved, since seats fill within minutes
Task 2 success	At least 25 percentage points more participants recover from the full-class error
Errors	At least 1 fewer error per task on average
SUS	At least a 10 point increase, or B crossing the commonly cited 68 point average

In HCI the stakes of a finding depend on the context. For registration, a keyboard user who cannot add a class at all is a total failure for that person, even if it barely moves an average. So I report effect sizes and confidence intervals next to every p-value, and I look at individual outcomes (like how many people could not finish at all on A) instead of only means. A result is only a strong argument for shipping B if it is statistically significant **and** clears the practical threshold.

2. Asymmetric transfer and carryover in my within-subjects design

Possible asymmetric carryover:

- **B → A (positive transfer):** If a participant uses B first, they learn the layout, what a successful add looks like, and that the full-class issue can be solved by choosing another section. When they later use A, they know what to look for and may try workarounds (such as Shift+Tab or tabbing through the whole page), which makes A look better than it really is.
- **A → B (negative transfer or frustration):** If a participant starts with A, they may hit keyboard dead ends and form a habit of over-tabbing or a belief that "the keyboard doesn't work here." That frustration can lower their SUS rating or slow them down at the start of B, making B look worse than it really is. It can also go the other way (contrast effect): after struggling on A, B might feel unusually good, inflating B's SUS.
- These two effects are **not** the same size. Knowing that a full-class problem can be solved is information that transfers strongly from B to A, while the confusion from A gives a person little useful knowledge to carry into B. That is what makes the transfer asymmetric.

How the counterbalancing isolates bias, mathematically: Let μ_A and μ_B be the true condition means, π the general practice effect of being in Period 2, and λ_A, λ_B the carryover from having used A or B first.

- Sequence group AB (S1, S2): Period 1 = μ_A, Period 2 = $\mu_B + \pi + \lambda_A$
  Difference (B − A) = $(\mu_B - \mu_A) + \pi + \lambda_A$
- Sequence group BA (S3, S4): Period 1 = μ_B, Period 2 = $\mu_A + \pi + \lambda_B$
  Difference (B − A) = $(\mu_B - \mu_A) - \pi - \lambda_B$

Because the groups are equal in size (8 each), averaging them gives:

Estimated effect = $(\mu_B - \mu_A) + (\lambda_A - \lambda_B) / 2$

- The **practice effect π cancels out completely**, because it is added in one group and subtracted in the other.
- **Symmetric carryover ($\lambda_A = \lambda_B$) also cancels.**
- **Asymmetric carryover does not cancel**; it leaves a bias of $(\lambda_A - \lambda_B) / 2$. Counterbalancing spreads order effects evenly; it cannot erase an effect that is bigger in one direction.

The second dimension of my Latin square (task set X vs. Y) works the same way: since each interface is paired with each task set equally often, any difficulty difference between X and Y cancels out of the A vs. B comparison.

How I handle the leftover asymmetric bias:

1. **Reduce it:** isomorphic task sets with different course codes so nothing is memorized, a neutral keyboard warm-up before Period 1 so general keyboard practice happens before data collection, and a 3 minute break between conditions.
2. **Detect it:** compare the sum (Period 1 + Period 2) between the AB and BA groups (the Grizzle carryover test), and include a Sequence term in the mixed model. A significant difference means $\lambda_A \neq \lambda_B$.
3. **Fallback:** if carryover is significant, I analyze only Period 1 data as a between-subjects comparison (8 vs. 8). Nobody has seen the other interface yet in Period 1, so carryover is impossible there. It has less power, but the result is unbiased.